



MDMC

Master in Data Management
and Curation

AREA
SCIENCE PARK
■■■■



Scientific Programming Environment

Lecture II: Bash

Jacopo Salvalaggio

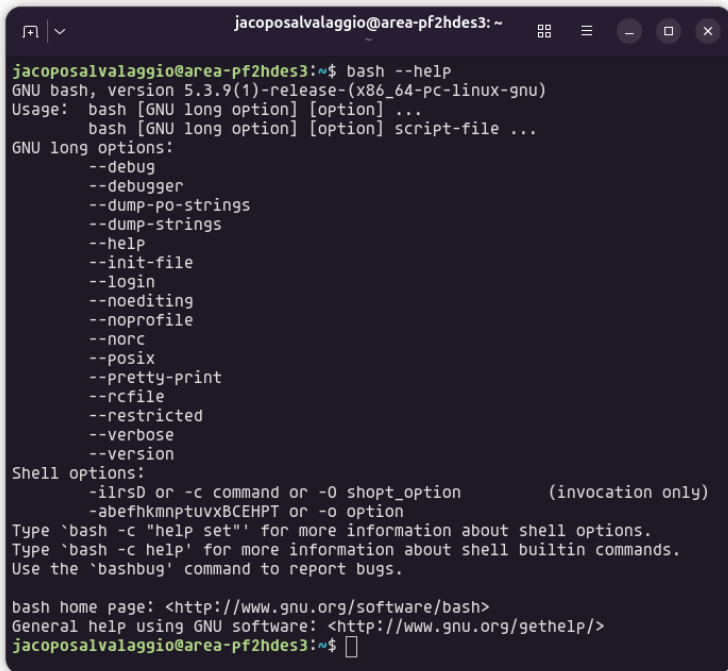
Special thanks to Gianfranco Gallizia



Image credit: globalsources.com



Goals



```
jacoposalvalaggio@area-pf2hdes3: ~  
jacoposalvalaggio@area-pf2hdes3:~$ bash --help  
GNU bash, version 5.3.9(1)-release-(x86_64-pc-linux-gnu)  
Usage: bash [GNU long option] [option] ...  
       bash [GNU long option] [option] script-file ...  
GNU long options:  
  --debug  
  --debugger  
  --dump-po-strings  
  --dump-strings  
  --help  
  --init-file  
  --login  
  --noediting  
  --noprofile  
  --norc  
  --posix  
  --pretty-print  
  --rcfile  
  --restricted  
  --verbose  
  --version  
Shell options:  
  -ilrsD or -c command or -O shopt_option      (invocation only)  
  -abefhkmnptuvxBCEHPT or -o option  
Type 'bash -c "help set"' for more information about shell options.  
Type 'bash -c help' for more information about shell builtin commands.  
Use the 'bashbug' command to report bugs.  
  
bash home page: <http://www.gnu.org/software/bash>  
General help using GNU software: <http://www.gnu.org/gethelp/>  
jacoposalvalaggio@area-pf2hdes3:~$
```

Objectives of the lecture:

- * Discover fundamental **shell commands**
- * Learn how to **navigate** a machine from the terminal
- * Group together commands in **bash scripts**

What is Bash?

Bash stands for "**Bourne Again Shell**" and is an **evolution** of the original UNIX shell.

Essentially, however, it does the same thing: provide and **interface between you and the computer** and allow you to run **commands**.

What are commands for?

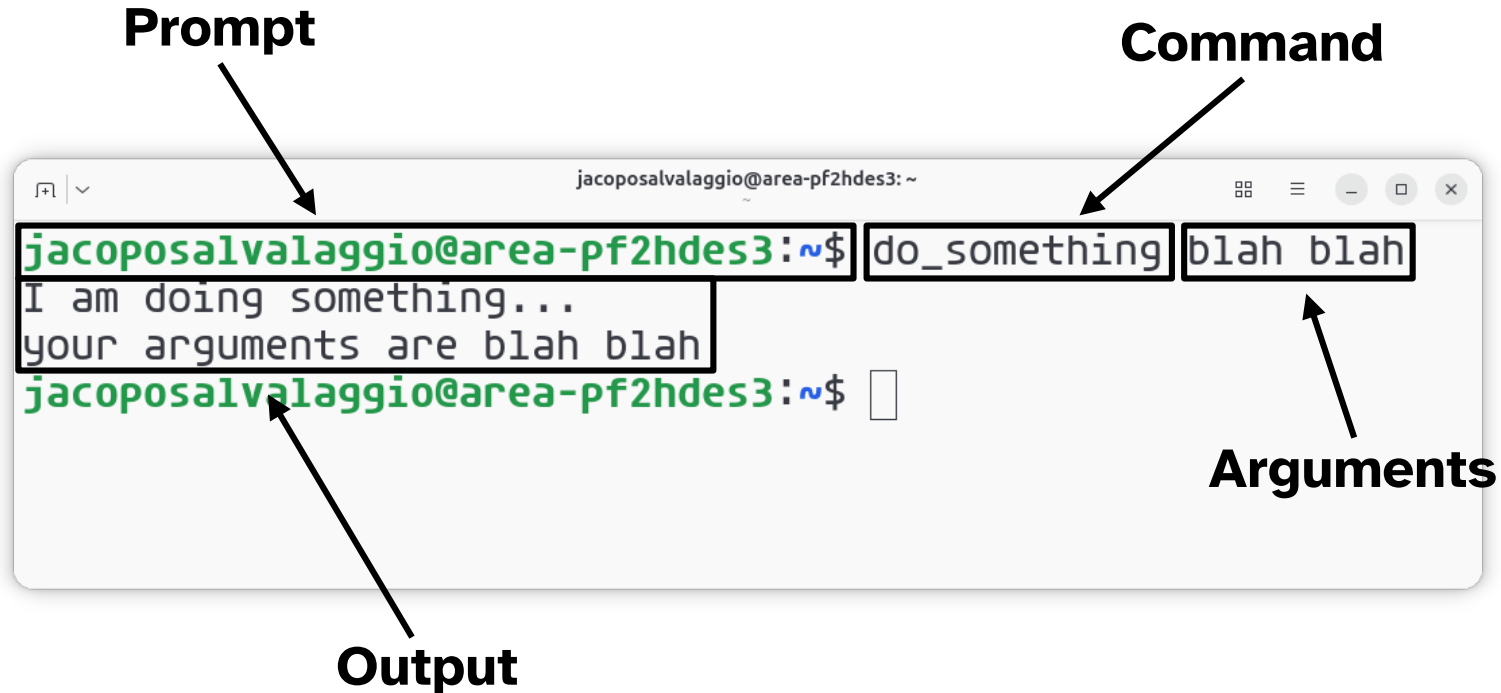
Commands on the UNIX shell can do anything you would do on a graphical user interface, for example:

- * **Navigate** around the filesystem
- * **Copy, move, delete** files
- * **Find** files
- * **Edit** files
- * ...



Image credit: [wecareanimalrescue.org](https://www.wecareanimalrescue.org)

Command in the shell



Getting help

The most basic commands are the ones that teach you **how to use** the others:

- * Manual: **man**
- * Manual page finder: **apropos**
- * Manual for bash builtin commands: **help**
- * Usually, you can ask for help in most command by adding **-h** or **--help** as argument

Getting help

```
jacoposalvalaggio@area-pf2hdes3: ~ -- man man
MAN(1) Manual pager utils MAN(1)

NAME
  man - an interface to the system reference manuals

SYNOPSIS
  man [man options] [[section] page ...] ...
  man -k [apropos options] regexp ...
  man -K [man options] [section] term ...
  man -f [whatis options] page ...
  man -l [man options] file ...
  man -w|-W [man options] page ...

DESCRIPTION
  man is the system's manual pager. Each page argument
  given to man is normally the name of a program, utility
  or function. The manual page associated with each of
  these arguments is then found and displayed. A section,
  if provided, will direct man to look only in that
  section of the manual. The default action is to search
  in all of the available sections following a pre-de-
  termined order.

Manual page man(1) line 1 (press h for help or q to quit)
```

\$ man man

```
jacoposalvalaggio@area-pf2hdes3: ~$ apropos --help
Usage: apropos [OPTION...] KEYWORD...

  -d, --debug                emit debugging messages
  -v, --verbose              print verbose warning messages
  -e, --exact                search each keyword for exact
                             match
  -r, --regex                interpret each keyword as a re
                             gex
  -w, --wildcard              the keyword(s) contain wildcar
                             ds
  -a, --and                  require all keywords to match
  -l, --long                  do not trim output to terminal
                             width
  -C, --config-file=FILE     use this user configuration fi
                             le
  -L, --locale=LOCALE        define the locale for this sea
                             rch
  -m, --systems=SYSTEM       use manual pages from other sy
                             stems
```

\$ apropos --help

Navigating the fs

Here are some commands useful to **move around files**:

- * `ls`, to **list** the contents of a directory
- * `cd`, to **change** the **directory** you are in
- * `pwd`, to **print** the path to your current **directory**
- * `mkdir`, to **create** a new **directory**

Show comparison with GUI...

Navigating the fs

```
jacoposalvalaggio@area-pf2hdes3: ~/open-webui
~ /open-webui

jacoposalvalaggio@area-pf2hdes3:~$ ls
Code          gopass_history.txt
Desktop       log_temp_boh.boh
Documents     nuova_rendicontazione_novalia.txt
Downloads     open-webui
Music         provaprova
Notes         rendicontazione_dettaglio_novalia.txt
Pictures      secrets_for_be.yaml
Public        snap
Python-3.11.15 temp
Templates     temp2
Videos        test.test
area

jacoposalvalaggio@area-pf2hdes3:~$ pwd
/home/jacoposalvalaggio

jacoposalvalaggio@area-pf2hdes3:~$ cd open-webui/
jacoposalvalaggio@area-pf2hdes3:~/open-webui$ pwd
/home/jacoposalvalaggio/open-webui

jacoposalvalaggio@area-pf2hdes3:~/open-webui$
```

ls lists files in alphabetical order by default (uppercase first)

If no argument is supplied it will list files in ./

pwd reveals the absolute path to the working directory

cd wants the target directory as an argument

Bash wildcards

Many bash commands accept **lists of files as input** argument. Instead of writing them one by one, you can create a list automatically **specifying a rule**.

Idea:

```
$ ls some_directory/  
file_1 file_2 file_3 something_else  
$ ls some_directory/"something starting with 'file'"  
file_1 file_2 file_3
```

Wildcard rules

Basic wildcard rules:

- * `*` generates a match with **any string** → `some*` will match both `something` and `someone`
- * `?` matches a **single character** → `som?` will match `some` but not `something`
- * `[abc]` matches the **single character to be either a, b, c** (the specific letters don't matter) → `som[aei]` matches `soma`, `some` and `somi`

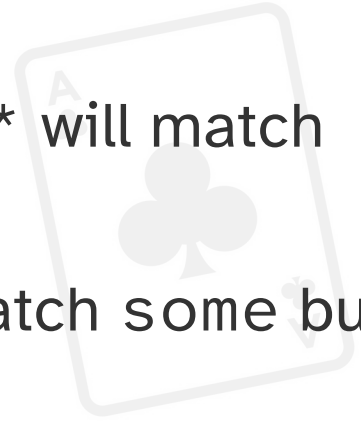


Image credits: Astrid Lindawati @ vecteezy.com

Navigating the fs

```
jacoposalvalaggio@area-pf2hdes3: ~/testing
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls
soma some someone something somi somo somu
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls som*
soma some someone something somi somo somu
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls some*
some someone something
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls som?
soma some somi somo somu
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls som[aei]
soma some somi
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls some[ot]*
someone something
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls som[a-z]
soma some somi somo somu
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls *
soma some someone something somi somo somu
jacoposalvalaggio@area-pf2hdes3:~/testing$
```

Notice how `*` also matches an empty string

The `[...]` wildcard also accepts ranges

Using `*` is like saying "match anything"

Handling files

Three basic commands allow you to move and remove files:

- * `cp` is used to **copy** files
- * `mv` to **move** and **rename** files
- * `rm` to **delete** files

Careful: there is no trash to recover after deleting a file!

Handling files

```
jacoposalvalaggio@area-pf2hdes3: ~/testing
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls
jacoposalvalaggio@area-pf2hdes3:~/testing$ cp ../temp ./copy_test
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls
copy_test
jacoposalvalaggio@area-pf2hdes3:~/testing$ cp ../temp ./temp
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls
copy_test  temp
jacoposalvalaggio@area-pf2hdes3:~/testing$ mv temp temp_renamed
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls
copy_test  temp_renamed
jacoposalvalaggio@area-pf2hdes3:~/testing$ rm copy_test temp_renamed
jacoposalvalaggio@area-pf2hdes3:~/testing$ ls
```

cp requires the source and destination file or the source file and destination directory

mv moves the source file to the destination, so it also works for renaming

rm deletes files without warning by default!

Printing text

There are various ways to print text on the terminal

- * **echo** prints out all arguments provided
- * **cat** prints the contents of one or more files
- * **head** and **tail** print the beginning and end of a file respectively
- * **less** lets you scroll up and down the contents of a file

Cue some examples...

Output redirection

Normally a command like `echo` and `cat` prints to **standard output**, that is it just prints lines on the terminal.

You can **redirect** this output **into a file**, so that it is saved somewhere.

In order to do so the `>` and `>>` operators are used: the first overwrites the destination file and the second appends to it.

Output redirection

```
jacoposalvalaggio@area-pf2hdes3: ~/testing
jacoposalvalaggio@area-pf2hdes3:~/testing$ echo "Hello, I am a simple text file" > simple_text_file.txt
jacoposalvalaggio@area-pf2hdes3:~/testing$ cat simple_text_file.txt
Hello, I am a simple text file
jacoposalvalaggio@area-pf2hdes3:~/testing$ echo "Hello, I am a very simple text file!" > simple_text_file.txt
jacoposalvalaggio@area-pf2hdes3:~/testing$ cat simple_text_file.txt
Hello, I am a very simple text file!
jacoposalvalaggio@area-pf2hdes3:~/testing$ echo "Hello, I am a simple text file" >> simple_text_file.txt
jacoposalvalaggio@area-pf2hdes3:~/testing$ cat simple_text_file.txt
Hello, I am a very simple text file!
Hello, I am a simple text file
jacoposalvalaggio@area-pf2hdes3:~/testing$
```

Using > has overwritten the original file (beware!)

Using >> has kept the original line and added a new one

Filters

Filters are commands that **select** something out of a piece of **input text**. Some examples are:

- * **cut** prints out selected **columns** of a file
- * **sort reorders** file lines
- * **uniq removes duplicate** lines
- * **grep** looks for lines **matching** a certain **pattern**

Cue some examples...

Pipelines

Pipelines represent one of the most **powerful** features of bash and the **philosophy of UNIX** put in practice.

They allow to take the **output** of a command and feed it **as the input** of the next in the chain.

This way, many **simpler commands** can be **combined** together.



Image credits: Luma Pulse @ pngall.com

Pipelines

Syntax for a pipeline is **command_1 | command_2**. You are not limited to two command but **can chain how many you like**.

Select lines from 6 to 10 of a file:

```
$ head -n 10 FILENAME | tail -n 5
```

Sort a file and then remove duplicate lines:

```
$ sort FILENAME | uniq
```

Numerically sort lines that contain the word "Error":

```
$ grep 'Error' FILENAME | sort -n
```

Regular expressions

grep matches a pattern to see if it appears in lines of a file. The way it does it is through **regular expressions** (regex), a more advanced version of **wildcards**.

- * . matches any character
- * * matches any number of iterations of the previous character
- * ^ and \$ specify the pattern should be at the beginning or end of the line respectively

grep examples

- * `grep 'some.*' FILE` will match any lines containing a word starting with some in the file
- * `grep '^In' FILE` will match all lines beginning with In
- * `grep 'hello' FILE` will match any line containing the string hello, anywhere
- * `grep -v 'NO' FILE` will match all lines that do not contain NO

← -v is called a **flag**

Bash variables

Variables represent named **memory** addresses where we **store** a certain **value**. That value can be retrieved with the **\$** operator. **Lists** are and **ordered** group of variables.

```
$ my_var="ciao"  
$ echo $my_var  
ciao  
$ my_var="hola"  
$ echo ${my_var}  
hola
```

```
$ my_list=("ciao" "hola")  
$ echo ${my_list[0]}  
ciao  
$ echo ${my_list[@]}  
ciao hola
```

Bash quotation

Single (') and double (") quotation marks have a different meaning in Bash. **Single quotation marks will not expand** any name variable inside.

```
$ my_var="ciao"  
$ echo '$my_var'  
$my_var  
$ echo "$my_var"  
ciao
```

Output as variable

It is possible to **redirect output** of a command **into a variable** with the **`$(...)`** syntax. The command between the brackets can also contain pipelines!

```
$ echo "Ciao ciao" > text_file.txt
$ echo "Hello hello" >> text_file.txt
$ first_word=$(head -n1 text_file.txt | cut -d' ' -f1)
$ echo $first_word
Ciao
```

Bash scripting

Now that we know some of the commands we can launch on bash, we can learn how to **combine** them together and **automate** them inside a **script**.

Bash supports standard constructs and logic as many other programming languages, such as **variables**, **lists**, **for** and **while loops** and **conditional** statements.

Every bash script begins with the line **#!/bin/bash**, that specifies what to use to execute it (remember, bash is also a command).

Executing a script

Once you have a script, you can run it in different ways:

- * `bash script_name`
- * `./script_name` if the file has execution permission
- * `source script_name` if you need exported variables to persist in your shell session



for loops

Example for loop with list:

```
my_list=(a b c d)
for element in ${my_list[@]}; do
    echo $element
done
```

Example for loop with explicit declaration:

```
for element in a b c d; do
    echo $element
done
```

for loops **repeat** a certain action **for each element of a list**.

It is possible to define the list beforehand or to declare the values to loop over directly.

Loop flow control

You can skip one step of a loop or break it entirely with the **continue** and **break** commands:

- * **continue** will **skip to the next repetition** of the loop without executing any remaining instruction left in that step
- * **break** will **close the loop** entirely, moving on to the next lines of code

if clause

If statements **check** some **condition** and run a block of code if that returns true and a different block of code otherwise

You can also
supply an
alternative —→
condition with
elif COND

A silly example:

```
if SOME_CONDITION; then
    echo "Condition true"
else
    echo "False!"
fi
# BTW, this is a comment
```

Conditions

We evaluate conditions with the **test** (`[ ... ]`) command (see also `[[ ... ]]`). The command supports many different kinds of tests, for example:

- * `[ -z VAR ]` → true if VAR is an empty string
- * `[ -d FILE ]` → true if FILE is a directory
- * `[ NUMBER -eq 0 ]` → true if NUMBER is equal to zero
- * `[ STRING = 'some' ]` → true if STRING is 'some'

Combining conditions

Conditions can be **reversed** or **combined** together.

- * `!` will invert the condition: `! [ -e FILENAME ]` is false then the file exists
- * `&&` will combine two conditions so that the result is true if both are: `[ -e FILENAME ] && [ FILENAME = 'ciao' ]` will not be true if the file exists but is not named 'ciao'
- * `|` will combine two conditions so that the result is true if at least one is

if and conditions

A slightly less silly example:

```
read your_name # Note: read saves user input
into the provided variable name
if [ $your_name = "Jacopo" ]; then
    echo "You are me!"
else
    echo "Who are you?"
fi
```

while loops

while (and `until`) loops **combine** the flow control of a **for** loop with the condition of an **if** statement.

An arguably even less silly example:

```
your_name="Nobody"
while [ $your_name != "Jacopo" ]; do
    echo "I am waiting for Jacopo"
    read -p "Write your name: " your_name
done
echo "Hi Jacopo, finally!"
```

Input arguments

It is possible to pass **arguments** as input to a script, just as you would do for a command. They are **numbered** as per the order they are provided in.

example.sh:

```
# First arg
echo $1
# Second arg
echo $2
# All args
echo $@
```

```
$ bash example.sh one
one

one
$ bash example.sh one two three four
one
two
one two three four
```

Functions

Functions allow to name blocks of code for convenience. They accept input arguments like scripts.

An example with functions:

```
greet() {  
    echo "Hello, ${1}. How are you?"  
}  
greet Jacopo  
greet Niccolo  
greet Tizio Caio
```

Exit status

Commands return an integer **exit status** when they finish. Conventionally, an exit status of **zero** means the code exited **successfully**. You can inspect the exit status of the last command run with **\$?**.

```
$ ls nothing
ls: cannot access 'nothing': No such file or directory
$ echo $?
2
$ cd ~/
$ echo $?
0
```

Exit status

You can add an exit status to your own code by providing an argument to the **exit** command.

Failure exit code example:

```
file_name=$1
if [ -z $file_name ]; then
    echo "No file name provided."
    exit 1
else
    cat $file_name
    exit 0
fi
```



Image credit: Abel+Imray

Thank you for your attention :)

Image credits for background
rainbow gradient: Image Colorful
Rainbow Pattern by sgagnonou812 at
Wallpapers.com